



Payload Decoding Guide

MeteoHelix IoT Pro Gen2 | How to parse the payloads
Firmware 1.02.023

1. What this document is for

This is a decoder-writer's guide for the MeteoHelix IoT Pro Gen2. It covers every LoRaWAN payload the station sends, in the order you need them to write a parser: how the bits are laid out, how each field converts to a physical value, and the three encodings that produce wrong readings if you decode them naively. Section 8 adds the alarm-settings downlink, which is bit-packed the same way, because building it is the same skill in reverse. A complete, tested JavaScript decoder is included in section 9.

The document is self-contained: everything needed to decode the station's uplinks and to build an alarm-settings command is here. The field layouts were taken from the firmware itself and the decoder in section 9 was verified against it, so where this guide disagrees with an older payload sheet, this guide describes what firmware 1.02.023 actually does.

1.1 What this document covers

Port	Message	Size	Direction	Section
1	Periodic measurement	16 B	uplink	4
3	Alarm event	10 B	uplink	5
2	Service message MH1S - device identity	14 B	uplink	6
2	Service message MH2S - alarm settings readback	14 B	uplink	7
4	Retransmission of a stored payload	16 or 10 B	uplink	4 / 5
10	Alarm-settings command	12 B	downlink	8

A payload on port 4 is a byte-for-byte replay of one the station sent earlier on port 1 or port 3, resent on request. It carries no extra header, so decode it by length: 16 bytes uses the periodic layout of section 4, 10 bytes the alarm layout of section 5. The index field inside identifies which original message it is.

Port 50 carries short replies to configuration commands. They are bare hex fields with no message identifier, so they can only be interpreted against the command that was just sent, and they are outside the scope of this guide.

2. The parsing model

Every payload is one continuous bitstream. Fields are packed most-significant-bit first, back to back, with no padding or alignment between them, so most fields straddle byte boundaries. Only the first field of the periodic payload happens to start and end on a byte.

Do not try to parse these payloads by slicing bytes. A byte-oriented parser works for the index field and then silently produces nonsense for everything after it. Read bit ranges.

2.1 Four steps

Step	What to do
1	Take the raw bytes and treat them as one bitstream, bit 1 being the most significant bit of byte 0
2	For each field in table order, read its width in bits, most significant bit first, into an unsigned integer - call it raw
3	If every bit of the field is set, the reading is INVALID. Emit null and stop processing that field. See 3.1
4	Otherwise the physical value is $\text{raw} * \text{resolution} + \text{minimum}$, using the resolution and minimum from the field table

2.2 A payload decoded by hand

Periodic payload 03B7DC305EE7D8D48003C1E006D6C1CC on port 1. Writing the 16 bytes out as 128 bits and cutting them at the field widths from section 4 gives:

Bits	Field	Bits read	raw	Value	Unit
1-8	Index	00000011	3	3	
9	Battery in slot	1	1	1	
10-23	Temperature	01101111101110	7150	21.5	deg C
24-31	T_min difference	00011000	24	1.2	deg C
32-39	T_max difference	00101111	47	2.35	deg C
40-48	Humidity	011100111	231	46.2	%
49-63	Pressure	110110001101010	27754	99385	Pa
64-74	Irradiation	01000000000	512	512	W/m2
75-84	Irr_min difference	0000111100	60	120	W/m2
85-94	Irr_max difference	0001111000	120	240	W/m2
95-106	Rain clicks	000000011011	27	27	clicks
107-116	Rain min time	0101101100	364	364	728/sqrt(s)
117-126	Rain intensity corr	0001110011	115	1.15	clicks
127	Heater activated	0	0	0	
128	Alarm sent	0	0	0	

Worked in full for the temperature field: bits 10-23 are 01101111101110 = 7150, and $7150 \times 0.01 + (-50.0) = 21.50$ deg C. The rain field is not finished at this point - 364 is not a number of seconds. See section 3.3.

3. Three things that break naive decoders

3.1 All-ones means "no reading", not "maximum"

When the firmware has no valid measurement - the sensor failed, the value was NaN, or it fell outside the encodable range - it does not clamp and it does not omit the field. It writes every bit of that field to 1.

A decoder that scales the all-ones pattern reports a plausible-looking extreme instead of a fault: 113.83 deg C for temperature, 102.2 % humidity, 111917.5 Pa, 2047 W/m2. These are the exact values a broken sensor produces. Test for all-ones BEFORE scaling and emit null.

Field	Bits	All-ones raw	Decodes to	Meaning
Temperature (periodic)	14	16383	113.83 deg C	No valid temperature
T_min / T_max difference	8	255	12.75 deg C	Difference not available
Humidity	9	511	102.2 %	No valid humidity
Pressure (periodic)	15	32767	111917.5 Pa	No valid pressure
Irradiation	11	2047	2047 W/m2	No valid irradiation
Irr_min / Irr_max difference	10	1023	2046 W/m2	Difference not available

The rule does not apply to counters, flags and identifiers - index, rain clicks, the heater and alarm bits, serial numbers and DevAddr - where an all-ones pattern is a legitimate value. The decoder in section 9 uses two different readers, `field()` and `plain()`, for exactly this reason.

3.2 The battery bit is not a voltage

The periodic payload carries one bit of battery information. It is 1 only when the battery voltage falls inside the single 200 mV slot selected by index % 5:

index % 5	Battery slot the bit reports on
0	3300 - 3500 mV
1	3500 - 3700 mV
2	3700 - 3900 mV
3	3900 - 4100 mV
4	4100 - 4300 mV

So a 1 means "the voltage is inside this payload's slot" and a 0 means "it is not" - which says nothing about which other slot it is in. To recover a voltage a server must watch five consecutive indices and note which one reported 1. Until then the battery state is unknown, and a device that has just joined cannot report a voltage at all for the first few uplinks.

If every one of five consecutive indices reports 0, the voltage is outside 3300-4300 mV entirely - treat that as a fault rather than as missing data.

3.3 Rain time is transmitted as $728 / \sqrt{t}$ (seconds)

The rain field is not an interval. The firmware transmits $728 / \sqrt{t}$ where t is the shortest interval between two tipping-bucket clicks, which puts the resolution where it matters - heavy rain, short intervals. A transmitted 0 means no two clicks were recorded, not a zero interval.

Quantity	Formula
Interval between clicks	$t_seconds = (728 / raw)^2$
Rain rate, from the interval	$rate_mm_h = 3600 / t_seconds * bucket_mm$
Rain rate, direct from raw	$rate_mm_h = 3600 * bucket_mm * raw^2 / 529984$

The direct form avoids the square root and is what the decoder uses; 529984 is 728 squared. bucket_mm is the rain gauge resolution in mm per click and is a property of the gauge fitted, not of the payload - the device never transmits it. Set it once in the decoder.

raw	Interval	Rate at 0.1 mm	Rate at 0.2 mm	Rate at 0.25 mm	mm/h per LSB (0.2 mm)
0	not measured	-	-	-	-
104	49.0 s	7.3	14.7	18.4	0.28
230	10.02 s	35.9	71.9	89.8	0.62
364	4.00 s	90.0	180.0	225.0	0.99
728	1.00 s	360.0	720.0	900.0	1.98
1023	0.51 s	710.9	1421.7	1777.2	2.78

Resolution degrades sharply at high rain rates: near the top of the range one LSB is worth about 2.8 mm/h with a 0.2 mm gauge. Do not present decoded heavy rain rates to more precision than that.



4. Port 1 - periodic measurement, 16 bytes

The routine uplink, sent every 2 to 10 minutes. Also appears on port 4 as a retransmission. Uses all 128 bits.

Bits	Field	W	Min	Res	Decoded unit	Notes
1-8	Index	8	0	1	-	Revolving 0-255. Identifies the payload for retransmission requests
9	Battery in slot	1	0	1	-	See 3.2 - not a voltage
10-23	Temperature	14	-50.0	0.01	deg C	Period average
24-31	T_min difference	8	0.0	0.05	deg C	Subtract from the average to get the period minimum
32-39	T_max difference	8	0.0	0.05	deg C	Add to the average to get the period maximum
40-48	Humidity	9	0.0	0.2	%	Period average
49-63	Pressure	15	30000	2.5	Pa	Period average
64-74	Irradiation	11	0	1	W/m2	Period average
75-84	Irr_min difference	10	0	2	W/m2	Subtract from the average. Always a multiple of 2
85-94	Irr_max difference	10	0	2	W/m2	Add to the average. Always a multiple of 2
95-106	Rain clicks	12	0	1	clicks	Lifetime revolving counter, NOT per period - see 4.1
107-116	Rain min time	10	0	1	-	Apply 3.3. 0 means not measured
117-126	Rain intensity corr	10	0	0.01	clicks	Fractional click carried over, 0.00-10.23
127	Heater activated	1	0	1	-	1 if the sensor heater ran during the period
128	Alarm sent	1	0	1	-	1 if an alarm payload was sent during the period

4.1 Rain clicks is a lifetime counter

The rain clicks field is the device's running total, not rainfall in the period. To get rainfall between two uplinks, subtract consecutive values and handle the 12-bit wrap at 4095. It can also be set to an arbitrary value by downlink command 121, and it resets to whatever was stored in EEPROM after a reboot - so a decrease does not always mean a wrap.

Rainfall between two payloads is $(\text{clicks_now} - \text{clicks_prev}) \bmod 4096$ multiplied by the gauge resolution in mm. The intensity correction field carries the fractional click not yet counted and can be added for a slightly more accurate total.

4.2 Reconstructing the period minimum and maximum

The minimum and maximum are transmitted as differences from the average, not as absolute values, which is what keeps them to 8 and 10 bits:

Quantity	Reconstruction
Period minimum temperature	$\text{temperature} - \text{t_min_difference}$
Period maximum temperature	$\text{temperature} + \text{t_max_difference}$
Period minimum irradiation	$\text{irradiation} - \text{irr_min_difference}$
Period maximum irradiation	$\text{irradiation} + \text{irr_max_difference}$

If the average is null because of the all-ones rule, the reconstructed minimum and maximum are meaningless - skip them rather than reporting an offset from an error value.

5. Port 3 - alarm, 10 bytes

Sent when a configured threshold is crossed, carrying instantaneous readings rather than period averages. Also appears on port 4 as a retransmission. Uses 77 of the 80 bits; bits 78-80 are zero padding, not a field.

Bits	Field	W	Min	Res	Decoded unit	Notes
1-5	Index	5	0	1	-	Revolving 0-31, a separate counter from the periodic index
6-8	Alarm type	3	0	1	-	0 temperature, 1 humidity, 2 pressure, 3 rain. 4-7 unused
9-19	Temperature	11	-50.0	0.05	deg C	Instantaneous. Coarser than the periodic field
20-28	Humidity	9	0	0.2	%	Instantaneous
29-43	Pressure	15	30000	3	Pa	Instantaneous. Resolution is 3 Pa, NOT 2.5 - see 5.1
44-53	Irradiation	10	0	2	W/m2	Instantaneous
54-65	Rain clicks	12	0	1	clicks	Lifetime revolving counter, as in 4.1
66-75	Rain interval	10	0	1	-	Apply 3.3
76-77	Debug flags	2	0	1	-	Always 0 in this firmware
78-80	(padding)	3	-	-	-	Zero fill, not a field

5.1 The alarm payload is not the periodic payload

Three fields share a name with the periodic payload but not an encoding. Temperature is 11 bits at 0.05 deg C here against 14 bits at 0.01; pressure is 3 Pa per step against 2.5; irradiation is 2 W/m2 per step against 1. Reusing the periodic decoder on a port 3 payload produces readings that look reasonable and are wrong.

5.2 Only one alarm type is reported

The firmware evaluates temperature, humidity, pressure and rain in that order and each match overwrites the previous one, so the type field reports the LAST condition matched: rain masks pressure, pressure masks humidity, humidity masks temperature. If a station reports a rain alarm you cannot conclude the temperature was in range - only that rain was also out of range. The other readings in the payload are still present and can be checked against the thresholds independently.

6. Port 2 - MH1S service message, 14 bytes

Device identity. Sent only in response to downlink command 120 with parameter 01. Uses all 112 bits.

MH1S and MH2S are both 14 bytes on port 2. Read the first 2 bits to tell them apart: 1 is MH1S, 2 is MH2S. Do not branch on length.

Bits	Field	W	Notes
1-2	Message type	2	Always 1 for MH1S. This is the discriminator
3-7	Hardware type	5	0 test, 1 MeteoHelix, 2 MeteoWind, 3 MeteoRain, 4 MeteoAG
8-10	FW major	3	Firmware version, first component
11-16	FW minor	6	Second component. Render zero-padded to 2 digits
17-24	FW patch	8	Third component. Render zero-padded to 3 digits
25-56	Sensor head serial	32	SHT45 serial. 0 means the sensor did not report a valid status
57-80	Solar serial	24	Always 0 - not implemented in this firmware
81-112	Device address	32	LoRaWAN DevAddr. Render as 8 hex digits

The three version components are separate fields, so 1, 2, 23 renders as 1.02.023 - zero-pad the minor to two digits and the patch to three to match how the station names its own version. Note that MH1S carries no measurement-period field - the uplink period cannot be read back over the air.

7. Port 2 - MH2S service message, 14 bytes

The alarm thresholds currently in effect. Sent only in response to downlink command 120 with parameter 02, and it is the only way to confirm that an alarm-settings downlink was accepted, because that command sends no reply of its own.

MH2S carries 87 bits of fields inside a 14-byte frame; bits 88-112 are zero padding. Older payload documentation describes this message as 12 bytes with a 9-bit debug-flags field at the end. Decode the first 87 bits and ignore the rest - and do not length-check for 12 bytes, or every frame will be rejected. See 10.2.

Bits	Field	W	Min	Res	Decoded unit	Notes
1-2	Message type	2	0	1	-	Always 2 for MH2S
3-11	All-alarms snooze	9	0	120	s	Hold-off after any alarm, 120-61320 s
12-13	Temp alarm enable	2	0	1	-	Bit mask, see 7.1
14-23	Temperature low	10	-50.0	0.1	deg C	Threshold that raises a low alarm
24-33	Temperature high	10	-50.0	0.1	deg C	Threshold that raises a high alarm
34-35	Humidity alarm enable	2	0	1	-	Bit mask, see 7.1
36-42	Humidity low	7	0	1	%	0-127
43-49	Humidity high	7	0	1	%	0-127
50-51	Pressure alarm enable	2	0	1	-	Bit mask, see 7.1
52-64	Pressure low	13	30000	10	Pa	30000-111910
65-77	Pressure high	13	30000	10	Pa	30000-111910
78-87	Rain click interval	10	0	1	-	Apply 3.3
88-112	(padding)	25	-	-	-	Zero fill, not a field

7.1 The enable fields are bit masks

Each of the three enable fields is two bits, and they are masks rather than an enumeration: bit 0 arms the low threshold and bit 1 arms the high threshold. Decoding them as an enum gets 3 wrong.

Value	Low threshold	High threshold	Meaning
0	off	off	Alarm disabled
1	armed	off	Low alarm only
2	off	armed	High alarm only
3	armed	armed	Both thresholds armed

The threshold values remain in the payload even when the corresponding alarm is disabled, so do not read a threshold as evidence that an alarm is active - check the enable field.

8. Port 10 - alarm-settings downlink, 12 bytes

This is the one downlink built the same way as the uplinks. Its 96 bits are the same 87 fields as MH2S (section 7) followed by a 9-bit debug field, filling 12 bytes exactly. If you can decode MH2S you can already encode this.

Property	Value
Port	10
Payload	12 bytes
Reply	None - see 8.2
Effect	Replaces all alarm thresholds at once. The station keeps them through a power cycle
Device reset	No

The station is a Class A LoRaWAN device: it only listens briefly after it transmits. A downlink queued on the network server is delivered after the station's next uplink, which can be up to the configured measurement period away - by default 10 minutes, plus up to 29 seconds of transmit spreading. Queue one command at a time and wait for it to be delivered.

Bits	Field	W	Min	Res	Valid range the firmware enforces
1-2	Type	2	0	1	Parsed, then discarded. Send 0
3-11	All-alarms snooze	9	0	120	120 - 61320 s
12-13	Temp alarm enable	2	0	1	0-3, bit mask as in 7.1
14-23	Temperature low	10	-50.0	0.1	-50.00 to +52.30 deg C
24-33	Temperature high	10	-50.0	0.1	-50.00 to +52.30 deg C
34-35	Humidity alarm enable	2	0	1	0-3, bit mask as in 7.1
36-42	Humidity low	7	0	1	0 - 127 %
43-49	Humidity high	7	0	1	0 - 127 %
50-51	Pressure alarm enable	2	0	1	0-3, bit mask as in 7.1
52-64	Pressure low	13	30000	10	30000 - 111910 Pa
65-77	Pressure high	13	30000	10	30000 - 111910 Pa
78-87	Rain click interval	10	0	1	728/sqrt form, see 3.3. 0 selects the 4.0 s default
88-96	Debug flags	9	0	1	Parsed, then discarded. Send 0

8.1 Encoding, and where it differs from decoding an uplink

Encoding is the parsing model of section 2 run backwards: for each field, $raw = round((value - minimum) / resolution)$, then append that raw value as the stated number of bits, most significant bit first.

The all-ones sentinel of section 3.1 does NOT apply to this payload. It is an uplink convention for reporting a failed sensor. Here an all-ones field is simply a large number, and a large number out of range gets the command rejected. Never write all-ones as a "not set" marker.

Two fields have no effect: the 2-bit type at the start and the 9-bit debug field at the end are both read by the station and then discarded. Send zeros in both. They exist only to make the structure line up with MH2S.

Validation is all-or-nothing and silent. If any single field falls outside the range in the table, the firmware discards the ENTIRE command and keeps the previous settings. There is no reply either way, so a rejected write looks exactly like a downlink that never arrived. Range-check before sending and confirm afterwards by requesting MH2S.

8.2 Confirming a write

Because the command sends no reply, the only confirmation is to request the MH2S service message - downlink command 120 with payload 02 - and compare the thresholds it reports against what you sent. The two layouts share their first 87 bits, so the comparison is field for field.

8.3 Cloning settings between devices

A useful consequence of the shared layout: take an MH2S uplink, keep the first 12 of its 14 bytes, and you have a valid port 10 command that applies those exact settings to another device. The two bytes dropped are the MH2S zero padding, and they land where the discarded debug field sits.

Step	Bytes
MH2S uplink received (14 B, port 2)	80BAA10800320000FFFAD8000000
First 12 bytes, sent as a port 10 downlink	80BAA10800320000FFFAD800
Same command built from scratch	00BAA10800320000FFFAD800

The two commands differ only in the leading 2 bits - the type field - which the firmware discards, so both apply identical settings. This was verified by running both through the firmware's own bit-extraction logic.

8.4 Worked example

Snooze 600 s; temperature alarm with both thresholds armed at -16.4 and +2.8 deg C; humidity and pressure alarms off; rain alarm at a 4.0 s click interval (raw 364):

Encoding	Value
Downlink (12 B, port 10)	00BAA10800320000FFFAD800
Base64 for the network server	ALqhCAAyAAD/+tgA

Field by field: snooze $600/120 = 5$; enable 3; low $(-16.4 + 50)/0.1 = 336$; high $(2.8 + 50)/0.1 = 528$; humidity enable 0, low 0, high 100; pressure enable 0, low $(30000-30000)/10 = 0$, high $(111910-30000)/10 = 8191$; rain 364; debug 0.

9. Reference decoder

The decoder below is supplied alongside this document as the file `meteohe lix_gen2_decoder.js`. Use that file rather than retyping from this page.

It is written in ES5 with no dependencies, so it pastes directly into a The Things Stack v3 or ChirpStack v4 payload-formatter box; both call `decodeUplink({bytes, fPort})`. Set `RAIN_BUCKET_MM` at the top to the resolution of the rain gauge fitted before using it.

This decoder was verified field by field against firmware 1.02.023 using the test vectors in section 9.1, including the sensor-failure case. Run them through your own decoder before trusting it.

9.1 Test vectors

Feed these through any decoder before trusting it. They are generated from an encoder that mirrors the firmware's bit packer.

Port	Payload (hex)	Expected
1	03B7DC305EE7D8D48003C1E006D6C1CC	index 3, 21.50 deg C (min 20.30, max 23.85), 46.2 %, 99385 Pa, 512 W/m2, 27 clicks, rain interval 4.00 s, correction 1.15, heater off, no alarm
1	077FFE0001FFFFFFFFFFFFC06C000000	index 7, temperature / humidity / pressure / irradiation all null (sensor failure), 27 clicks, rain interval null
3	285E3C9B7EC1600DAD80	index 5, temperature alarm, -12.35 deg C, 91.4 %, 100626 Pa, 88 W/m2, 27 clicks, rain interval 4.00 s
2	4242170EC999480000002700BE44	MH1S, MeteoHelix, firmware 1.02.023, sensor serial 248093000, DevAddr 2700BE44
2	80BAA10800320000FFFAD8000000	MH2S, snooze 600 s, temperature alarm both thresholds armed at -16.4 and 2.8 deg C, humidity and pressure alarms off, rain interval 4.00 s
4	03B7DC305EE7D8D48003C1E006D6C1CC	Same as the first row, flagged as a retransmission
10 dwn	00BAA10800320000FFFAD800	Alarm-settings downlink, section 8. Same settings as the MH2S row above. Pass to <code>decodeAlarmDownlink()</code> , not <code>decodeUplink()</code>

9.2 Source

```
// =====
// MeteoHelix IoT Pro Gen2 - LoRaWAN uplink payload decoder
// Barani Design Technologies
//
// For firmware 1.02.023. See the Payload Decoding Guide for the field
// layouts and the reasoning behind them.
//
// Works as-is as a TTN v3 / ChirpStack v4 JavaScript codec: paste the whole
// file into the payload-formatter box. decodeUplink({bytes, fPort}) is the
// entry point both stacks call.
//
// Ports: 1 periodic (16 B), 2 service MH1S/MH2S (14 B), 3 alarm (10 B),
//        4 retransmission (16 or 10 B), 50 command reply (raw hex).
//
// decodeAlarmDownlink(bytes) is also exported. It parses the 12-byte port 10
// alarm-settings DOWNLINK, for checking a command before sending it or
// reading one back from a log. TTN and ChirpStack never call it themselves.
// =====

// RAIN_BUCKET_MM: rain gauge resolution in mm per click - set to match the
// gauge fitted (0.1, 0.2 or 0.25).
var RAIN_BUCKET_MM = 0.2;

// ----- bit reader
function Reader(bytes) {
  this.b = bytes;
  this.pos = 0;
}
Reader.prototype.bits = function (n) {
  var v = 0;
```

```

    for (var i = 0; i < n; i++) {
        var byteNum = (this.pos + i) >> 3;
        var bitNum = 7 - ((this.pos + i) & 7);
        v = v * 2 + ((this.b[byteNum] >> bitNum) & 1);
    }
    this.pos += n;
    return v;
};

// Read a field and scale it. Returns null when every bit is set, which the
// firmware uses to mean "no valid reading" - not a maximum value.
function field(r, bits, min, res) {
    var raw = r.bits(bits);
    if (raw === Math.pow(2, bits) - 1) return null;
    return round4(raw * res + min);
}

// Same, but the all-ones pattern is a legitimate value (counters, flags, IDs).
function plain(r, bits, min, res) {
    var raw = r.bits(bits);
    if (min === undefined) return raw;
    return round4(raw * res + min);
}

function round4(x) {
    return Math.round(x * 10000) / 10000;
}

// 728/sqrt(t) -> seconds between rain clicks. 0 means "not measured".
function rainSeconds(raw) {
    if (raw === 0) return null;
    return round4((728 / raw) * (728 / raw));
}

// Rain rate in mm/h straight from the transmitted number:
// rate = 3600 * bucket * raw^2 / 728^2
function rainRate(raw, bucketMm) {
    if (raw === 0) return null;
    return round4((3600 * bucketMm * raw * raw) / 529984);
}

// ----- port 1 and 4
function decodePeriodic(r) {
    var o = {};
    o.index = plain(r, 8);
    o.batteryInSlot = plain(r, 1);
    o.temperature_C = field(r, 14, -50.0, 0.01);
    o.temperatureMinDiff_C = field(r, 8, 0.0, 0.05);
    o.temperatureMaxDiff_C = field(r, 8, 0.0, 0.05);
    o.humidity_pct = field(r, 9, 0.0, 0.2);
    o.pressure_Pa = field(r, 15, 30000, 2.5);
    o.irradiation_Wm2 = field(r, 11, 0, 1);
    o.irradiationMinDiff_Wm2 = field(r, 10, 0, 2);
    o.irradiationMaxDiff_Wm2 = field(r, 10, 0, 2);
    o.rainClicks = plain(r, 12);
    var rt = plain(r, 10);
    o.rainMinInterval_s = rainSeconds(rt);
    o.rainPeakRate_mm_h = rainRate(rt, RAIN_BUCKET_MM);
    o.rainIntensityCorrection = plain(r, 10, 0, 0.01);
    o.heaterActivated = plain(r, 1);
    o.alarmSent = plain(r, 1);

    // Absolute period minimum and maximum, reconstructed from the differences.
    if (o.temperature_C !== null) {
        if (o.temperatureMinDiff_C !== null)
            o.temperatureMin_C = round4(o.temperature_C - o.temperatureMinDiff_C);
        if (o.temperatureMaxDiff_C !== null)
            o.temperatureMax_C = round4(o.temperature_C + o.temperatureMaxDiff_C);
    }
    if (o.irradiation_Wm2 !== null) {
        if (o.irradiationMinDiff_Wm2 !== null)
            o.irradiationMin_Wm2 = o.irradiation_Wm2 - o.irradiationMinDiff_Wm2;
        if (o.irradiationMaxDiff_Wm2 !== null)
            o.irradiationMax_Wm2 = o.irradiation_Wm2 + o.irradiationMaxDiff_Wm2;
    }
    // One payload cannot give a voltage - see the battery section of the guide.
    o.batteryHint_mV = o.batteryInSlot
        ? { min: 3300 + 200 * (o.index % 5), max: 3300 + 200 * (o.index % 5) + 200 }
        : null;
    return o;
}

// ----- port 3 and 4
var ALARM_TYPE = ['temperature', 'humidity', 'pressure', 'rain'];

function decodeAlarm(r) {
    var o = {};

```

```

o.index = plain(r, 5);
var t = plain(r, 3);
o.alarmTypeValue = t;
o.alarmType = ALARM_TYPE[t] !== undefined ? ALARM_TYPE[t] : 'reserved';
o.temperature_C = field(r, 11, -50.0, 0.05);
o.humidity_pct = field(r, 9, 0, 0.2);
o.pressure_Pa = field(r, 15, 30000, 3);
o.irradiation_Wm2 = field(r, 10, 0, 2);
o.rainClicks = plain(r, 12);
var rt = plain(r, 10);
o.rainInterval_s = rainSeconds(rt);
o.rainRate_mm_h = rainRate(rt, RAIN_BUCKET_MM);
o.debugFlags = plain(r, 2);
return o;
}
// ----- port 2
var DEVICE_TYPE = ['test', 'MeteoHelix', 'MeteoWind', 'MeteoRain', 'MeteoAG'];

function decodeMh1s(r) {
  var o = { messageType: 'MH1S' };
  var hw = plain(r, 5);
  o.hardwareTypeValue = hw;
  o.hardwareType = DEVICE_TYPE[hw] !== undefined ? DEVICE_TYPE[hw] : 'unknown';
  var maj = plain(r, 3), min = plain(r, 6), pat = plain(r, 8);
  o.firmware = maj + '.' + pad2(min) + '.' + pad3(pat);
  o.sensorSerial = plain(r, 32);
  o.solarSerial = plain(r, 24); // always 0 in this firmware
  o.devAddr = hex8(plain(r, 32));
  return o;
}

function decodeMh2s(r) {
  var o = { messageType: 'MH2S' };
  o.allAlarmsSnooze_s = plain(r, 9, 0, 120);
  o.temperatureAlarm = alarmMode(plain(r, 2));
  o.temperatureLow_C = plain(r, 10, -50.0, 0.1);
  o.temperatureHigh_C = plain(r, 10, -50.0, 0.1);
  o.humidityAlarm = alarmMode(plain(r, 2));
  o.humidityLow_pct = plain(r, 7);
  o.humidityHigh_pct = plain(r, 7);
  o.pressureAlarm = alarmMode(plain(r, 2));
  o.pressureLow_Pa = plain(r, 13, 30000, 10);
  o.pressureHigh_Pa = plain(r, 13, 30000, 10);
  var rt = plain(r, 10);
  o.rainClickInterval_s = rainSeconds(rt);
  o.rainAlarmRate_mm_h = rainRate(rt, RAIN_BUCKET_MM);
  // Remaining 25 bits are zero padding - do not read them.
  return o;
}
// ----- port 10 downlink
// The alarm-settings DOWNLINK (12 bytes). Not an uplink - this is here so a
// backend can check what it is about to send, or read back a logged command.
// Its 96 bits are the same 87 fields as MH2S plus a 9-bit debug field; the
// firmware parses and then discards both the type and the debug field.
//
// Note the all-ones sentinel of the uplinks does NOT apply here. This is a
// command: out-of-range values are rejected by the firmware, not flagged.
function decodeAlarmDownlink(bytes) {
  if (bytes.length !== 12) {
    return { errors: ['alarm downlink needs 12 bytes, got ' + bytes.length] };
  }
  var r = new Reader(bytes), o = {};
  o.typeField = plain(r, 2); // discarded by the firmware
  o.allAlarmsSnooze_s = plain(r, 9, 0, 120);
  o.temperatureAlarm = alarmMode(plain(r, 2));
  o.temperatureLow_C = plain(r, 10, -50.0, 0.1);
  o.temperatureHigh_C = plain(r, 10, -50.0, 0.1);
  o.humidityAlarm = alarmMode(plain(r, 2));
  o.humidityLow_pct = plain(r, 7);
  o.humidityHigh_pct = plain(r, 7);
  o.pressureAlarm = alarmMode(plain(r, 2));
  o.pressureLow_Pa = plain(r, 13, 30000, 10);
  o.pressureHigh_Pa = plain(r, 13, 30000, 10);
  var rt = plain(r, 10);
  o.rainClickInterval_s = rt === 0 ? 4.0 : rainSeconds(rt); // 0 selects the default
  o.rainAlarmRate_mm_h = rainRate(rt, RAIN_BUCKET_MM);
  o.debugField = plain(r, 9); // discarded by the firmware
  return { data: o, warnings: rangeCheck(o) };
}

// The firmware validates all-or-nothing: one field out of range and the whole
// command is discarded, silently. Check before sending.
function rangeCheck(o) {
  var w = [];
  if (o.allAlarmsSnooze_s < 120 || o.allAlarmsSnooze_s > 61320)
    w.push('snooze ' + o.allAlarmsSnooze_s + ' s outside 120-61320');
  if (o.temperatureLow_C < -50 || o.temperatureLow_C > 52.3)

```

```

    w.push('temperature low outside -50.00..52.30');
    if (o.temperatureHigh_C < -50 || o.temperatureHigh_C > 52.3)
        w.push('temperature high outside -50.00..52.30');
    if (o.humidityLow_pct > 127) w.push('humidity low above 127');
    if (o.humidityHigh_pct > 127) w.push('humidity high above 127');
    if (o.pressureLow_Pa < 30000 || o.pressureLow_Pa > 111910)
        w.push('pressure low outside 30000-111910');
    if (o.pressureHigh_Pa < 30000 || o.pressureHigh_Pa > 111910)
        w.push('pressure high outside 30000-111910');
    if (w.length) w.push('THE WHOLE COMMAND WOULD BE DISCARDED by the device');
    return w;
}

// Bit 0 arms the low threshold, bit 1 arms the high threshold.
function alarmMode(v) {
    return { value: v, low: (v & 1) === 1, high: (v & 2) === 2 };
}

function pad2(n) { return (n < 10 ? '0' : '') + n; }
function pad3(n) { return (n < 10 ? '00' : n < 100 ? '0' : '') + n; }
function hex8(n) {
    var s = (n >>> 0).toString(16).toUpperCase();
    while (s.length < 8) s = '0' + s;
    return s;
}

// ----- dispatch
function decodeUplink(input) {
    var b = input.bytes, port = input.fPort;
    var r = new Reader(b);
    var data, warnings = [];

    if (port === 1) {
        if (b.length !== 16) return err('port 1 needs 16 bytes, got ' + b.length);
        data = decodePeriodic(r);
    } else if (port === 3) {
        if (b.length !== 10) return err('port 3 needs 10 bytes, got ' + b.length);
        data = decodeAlarm(r);
    } else if (port === 4) {
        // A replay of a port 1 or port 3 payload. Length is the only discriminator.
        if (b.length === 16) { data = decodePeriodic(r); data.retransmitted = true; }
        else if (b.length === 10) { data = decodeAlarm(r); data.retransmitted = true; }
        else return err('port 4 needs 16 or 10 bytes, got ' + b.length);
    } else if (port === 2) {
        // MH1S and MH2S are both 14 bytes - the leading 2 bits tell them apart.
        if (b.length !== 14) return err('port 2 needs 14 bytes, got ' + b.length);
        var type = r.bits(2);
        if (type === 1) data = decodeMh1s(r);
        else if (type === 2) data = decodeMh2s(r);
        else return err('unknown service message type ' + type);
    } else if (port === 50) {
        // Configuration command replies. Correlate with the command just sent -
        // these carry no command ID.
        return { data: { commandReply: bytesToHex(b) },
            warnings: ['port 50 reply - identify it from the command you sent'] };
    } else {
        return err('unhandled port ' + port);
    }
    return { data: data, warnings: warnings };
}

function err(m) { return { data: {}, errors: [m] }; }

function bytesToHex(b) {
    var s = '';
    for (var i = 0; i < b.length; i++) {
        var h = b[i].toString(16).toUpperCase();
        s += (h.length < 2 ? '0' : '') + h;
    }
    return s;
}

// Node/CommonJS export - harmless in TTN and ChirpStack, which ignore it.
if (typeof module !== 'undefined') { module.exports = { decodeUplink: decodeUplink, decodeAlarmDownlink: decodeAlarmDownlink }; }
```

10. Migrating an existing decoder

If you already decode an earlier MeteoHelix, or built a decoder from an older payload sheet, these are the points where that decoder will be wrong against firmware 1.02.023. Everything else - field positions and widths - is unchanged.

10.1 Invalid readings are probably being published as real ones

Older payload sheets scale every field unconditionally, so a decoder written from one converts the all-ones pattern into a physical value instead of recognising it as a fault. A station with a failed humidity sensor then reports a steady 102.2 %, and a failed temperature sensor reports 113.83 deg C, indefinitely.

This is the most consequential difference and the easiest to miss, because nothing looks broken - the numbers are in range and arrive on schedule. Implement the all-ones check of section 3.1 before anything else.

10.2 MH2S is 14 bytes, not 12

	Older documentation	Firmware 1.02.023
Frame length	12 bytes	14 bytes
Debug flags	9 bits at 88-96	Not transmitted; bits 88-112 are zero padding
Effect on a decoder	Rejects every real frame if it length-checks for 12 bytes	Field values for bits 1-87 are identical either way

Accept 14 bytes, decode the first 87 bits, and ignore the rest. If your decoder reads a debug-flags value from bits 88-96 it will always read zero, which is harmless, but the length check is not.

10.3 Smaller points

Item	Note
Alarm payload debug flags	Two bits (76-77), not four. The field is always zero in this firmware, so decoded values agree either way and only the bit count differs
MHOS service message	An older GPS and inclination service message - latitude, longitude, fix status, satellites, HDOP, inclination axes - appears in some earlier documentation. It is NOT implemented and the station never sends it. Do not write a decoder for it
Alarm-settings downlink	Unchanged. Its 96-bit layout is the same as in earlier documentation and is given in full in section 8

11. Decoder checklist

#	Check
1	Read bit ranges, not bytes
2	Test every measurement field for all-ones and emit null - do not scale it
3	Do not apply the all-ones rule to counters, flags, serials or DevAddr
4	Branch port 2 on the leading 2 bits, not on length - MH1S and MH2S are both 14 bytes
5	Accept 14 bytes for MH2S and decode only the first 87 bits
6	Use the port 3 resolutions for port 3 - temperature 0.05, pressure 3, irradiation 2
7	Convert rain with $(728/raw)^2$ and treat raw 0 as not measured
8	Set the rain bucket size to match the gauge; it is not in the payload

#	Check
9	Treat rain clicks as a lifetime counter and difference it, handling the 4095 wrap
10	Collect five consecutive indices before reporting a battery voltage
11	Decode port 4 by length: 16 bytes periodic, 10 bytes alarm
12	Expect one alarm type per event even when several thresholds were crossed
13	When ENCODING the port 10 downlink, range-check every field first - one bad field silently discards the whole command, and never use all-ones as a "not set" marker there

